

★ 3 IMPORTANT THINGS

ACTIVATION FUNCTION

↓
forward propagation

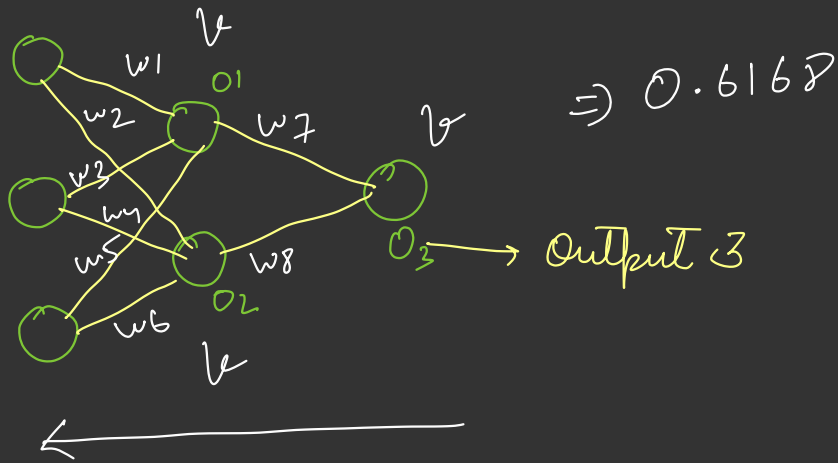
LOSS FUNCTION

OPTIMIZERS

Backward propagation

BACKWARD PROPAGATION

$$\begin{aligned} \hat{y} &= 0.6168 \\ y &= 1 \end{aligned} \rightarrow \begin{aligned} \text{Log loss} &= [y \times \log(\hat{y}) \\ &+ (1-y) \times \log(1-\hat{y})] \\ &= 0.494 \end{aligned}$$



Loss = 0.494 $\downarrow \downarrow$
 we have to lower the Loss.

① UPDATE THE WEIGHTS

$$w_{\text{new}} = w_{\text{old}} - \underbrace{\eta}_{\text{learning rate}} \frac{dL}{dw_{\text{old}}} \quad L = \text{Loss}$$

$$w_{7\text{ new}} = w_{7\text{ old}} - \eta \left[\frac{dL}{dw_{7\text{ old}}} \right] \Rightarrow \frac{dL}{dO_3} \times \frac{dO_3}{dw_{7\text{ old}}}$$

$$w_{1, \text{new}} = w_{1, \text{old}} - \eta \left[\frac{dL}{dw_{1, \text{old}}} \right] \Rightarrow \frac{dL}{dO_3} \times \frac{dO_3}{dO_1} \times \frac{dO_1}{dw_{1, \text{old}}}$$

$$w_{6, \text{new}} = w_{6, \text{old}} - \eta \left(\right) \frac{dL}{dw_{6, \text{old}}} = \frac{dL}{dO_3} \times \frac{dO_3}{dO_2} \times \frac{dO_2}{dO_6}$$

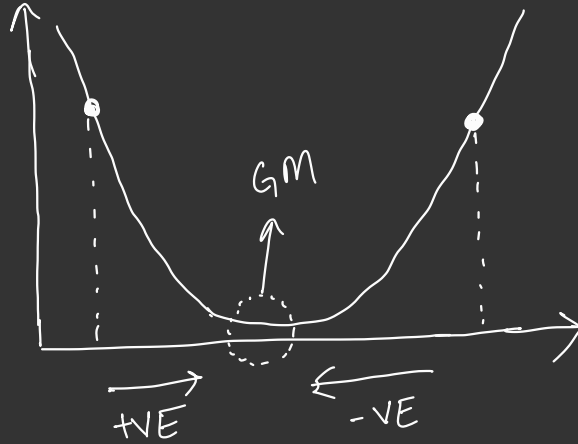


$$w_1, w_2, w_3, w_4, w_5, w_6 \Rightarrow 0.432 \quad \text{LOSS}$$

$$\begin{array}{c} \Downarrow \\ \text{changed} \end{array} \Rightarrow 0.349 \quad \text{LOSS}$$

★ Chain rule of derivatives...

AFTER THIS WE USE OPTIMIZERS LIKE GRADIENT DESCENT



1 epoch = 1 Forward Propagation
+
1 Backward Propagation.

→ 62%

2nd epoch → 69%

⋮

20th epoch → 94%

ACTIVATION FUNCTION ↴

① SIGMOID ACTIVATION FUNCTION

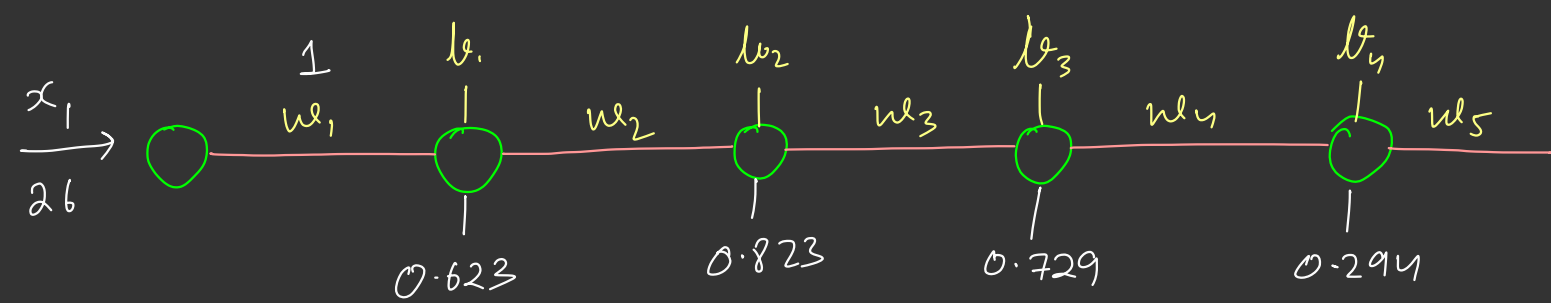
$$\hookrightarrow \sigma = \frac{1}{1 + e^{-z}}$$

A mathematical rule applied to a neuron's weighted input sum. It decides whether the neuron should pass or fire information.

↳ Vanishing gradient problem.

(happens during back propagation.)

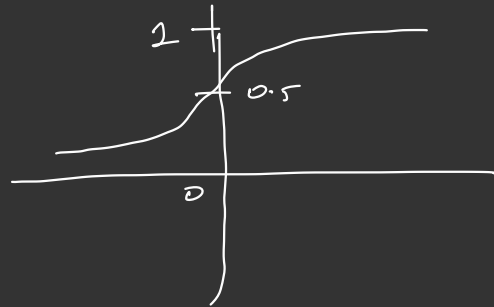
The vanishing gradient problem happens during back propagation when gradients become so tiny that the earlier layers of a deep neural network stop learning.



$$\sigma \left[(26 \times 1) + b \right] = \frac{1}{1 + e^{-z}} \Rightarrow \sigma = \frac{1}{1 + e^{-(26 \times 1) + b}}$$

$\Downarrow$
 0.623

$$w_{\text{new}} = w_{\text{old}} - \eta \left[\frac{d \text{ Loss}}{d w_{\text{old}}} \right]$$



$$\Rightarrow \frac{dL_{\text{loss}}}{dw_{1,\text{old}}} = \frac{dL_{\text{loss}}}{do_4} \times \frac{do_4}{do_3} \times \frac{do_3}{do_2} \times \frac{do_2}{do_1} \times \frac{do_1}{dw_1}$$

TYPE OF ACTIVATION FUNCTION

① Linear Activation function.

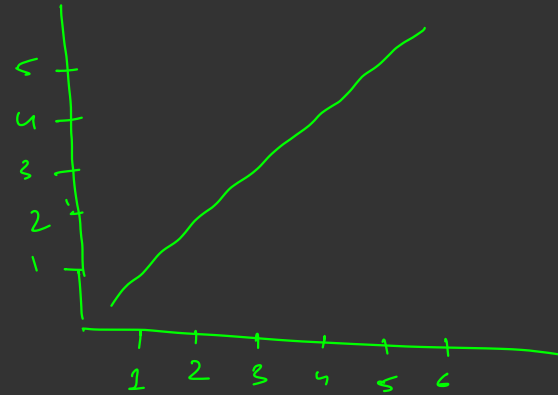
A linear activation function is the simplest type of activation function. It basically means

$$f(x) = x$$

so, the output is the same as the input.

$$[\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3]$$

If we use linear activation in all layers, the whole network becomes just a linear model, no matter how many layers add.



② Sigmoid Activation function

The sigmoid function is an S shaped curve that squashes any real number into a range between 0 and 1.

- The limitation is vanishing gradient problem for very large or very very small inputs, the gradient becomes almost 0, which slows learning.

